Lecture 4: Boolean/Digital logic

Lui Holder-Pearson

July 23, 2026

Learning Goals

- Analyse and write logic functions.
- Use mathematical tools to simplify logic functions.
- Apply Karnaugh maps to analyse and simplify logic functions.
- Implement logic functions as logic circuits.

Further Reading

ENCE361 - Next year
ENEL373 - Next year
Rizzoni and Kearns. *Principals and Applications of Electrical Engineering*, Chapter 13.3
https://www.youtube.com/watch?v=JQBRzsPhw2w - Organic Chemistry Tutor Channel, good explanation from low-level, including creation from transistors.

Version: 2.1

1 Logic variables

Recall:

Representation		
Numerical value:	0	1
Logical value:	`FALSE`	`TRUE`
Voltage:	Low	High
Presence:	Absent	Present
Switch state:	Open	Closed.

The values can be stored within logic variables.

1.1 Logical functions

We are quite familiar with the use of mathematical functions, where we have a numerical input and a numerical outputs, for example $f(x) = 2 \times x$, or functions of multiple variables, such as $f(x, y) = x^2 + y^3$.

Similarly, we have logical functions, where we have logical inputs and outputs. We have already been exposed to `NOT` and `OR`.

Truth tables are an effective method of showing logic functions:

Inputs	Outputs(z)			
	`AND`	`OR`	`NAND`	`NOR`
0, 0	0	0	1	1
0, 1	0	1	1	0
1, 0	0	1	1	0
1, 1	1	1	0	0

There are three **fundamental logical functions**:

Function	Mathematical function	C representation	Truth table
NOT:			
AND:			
OR:			

They are also the ones we can make relatively easily out of transistors:

These three logic functions are called the fundamental functions because every logical function, including functions of more than two variables, can be written in terms of only *three fundamental functions*. When this is done so, it is said that the function is in *canonical form*.

To understand how this is the case, it is important to understand the algebraic properties of logical variables.

1.2 Algebraic properties

- Closure:
- Identity:
- Associative:
- Commutative:
- Distributive:
- Annihilation:
- Adsorption:
- Idempotence:
- Double negation:
- Or-complement:
- And-complement:

Example:

Simplify the following expression:

$$f(a, b) = b + a \cdot (\bar{a} + b)$$

1.3 De Morgan's Laws

$$\overline{a + b} = \overline{a} \cdot \overline{b}$$

$$\overline{a \cdot b} = \overline{a} + \overline{b}$$

Therefore, it is possible to use a single function (`NAND`) to represent out three fundamental functions:

$$\bar{a} =$$

$$a \cdot b =$$

$$a + b =$$

Extra question: Do the same with `NOR`!

2 Karnaugh maps

Karnaugh maps are a graphical tool used to simplify logic functions. They can be used on functions with up to four variables.

A Karnaugh map is also known as a K-map

K-maps are a two dimensional representation of the truth table, where inputs are put along the axes. Specifically, the Karnaugh map must ordered such that there is always only a single variable change between each adjacent cell. Note this is different from how we would normally write a truth table (where the order doesn't matter beyond readability).

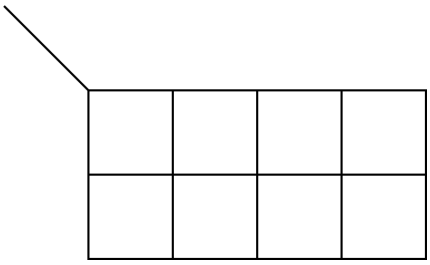


Figure 1: Three-variable K-map.

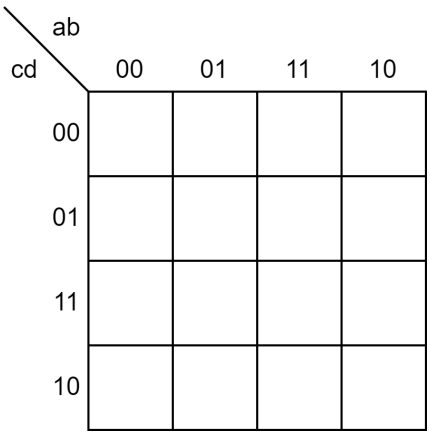
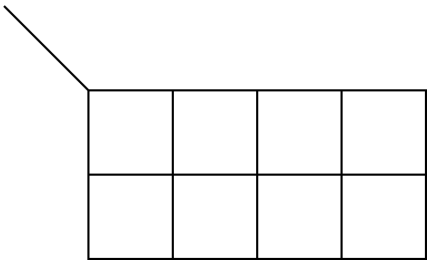


Figure 2: Four-variable K-map.

1. Expressions of up to four Boolean variables are able to be analysed.
2. Each cell in the K-map corresponds to a specific minterm.
3. Cells which share a border (i.e. horizontally and vertically adjacent cells but **not** diagonally adjacent cells) differ by a single variable.
4. Groups of neighbouring 1s in the K-map can be combined to give simplified expressions.
5. Group size is always 2^n: 2,4,8,16 variables.
6. All cells of the groups must be 1 for entire rectangles of size $2^{(0-3)} \times 2^{(0-3)}$
7. Groups can overlap
8. Groups can wrap around the boundaries of the K-map.

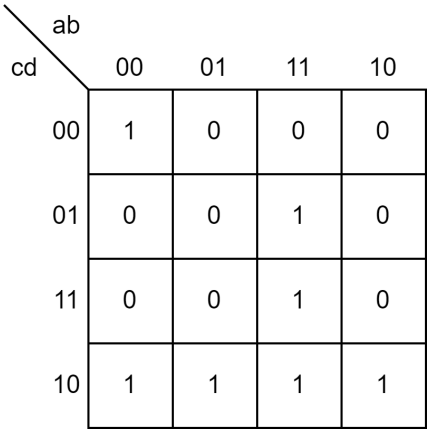
Simplification using a K-map:

Each group of variables can be represented as a single expression. Because each single movement vertically or horizontally will differ by a single variable, the group will not be dependant on the variable (or variables) that *does* change across the group. For the above example:



Or a more complex example:

$$f(a,b,c,d) = \bar{a}\bar{b}\bar{c}\bar{d} + \bar{a}\bar{b}c\bar{d} + \bar{a}bc\bar{d} + a\bar{b}c\bar{d} + abc\bar{d} + ab\bar{c}d + abcd$$



3 Logic circuit design

We now have the required tools to implement a logic circuit from a desired behaviour. This process is done with a simple step-wise process

1. Define the function using a *truth table*.
2. Convert the truth table to a *Boolean expression*.
3. *Simplify* using Boolean algebra.
4. Verify the expression using a *Karnaugh map*
5. Map Boolean operations to *logic gates*.

~~Also known as a K-map, named after Maurie Karnaugh, who introduced it in 1953 (and is actually still alive at the age of 98!!).~~

3.1 Boolean expression

The conversion from truth table to boolean expression is done by looking at each combination of inputs for which the output is 1 `(TRUE)`. Specifically, the function is written as a sum of products (SOP):

a	b	c	z
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

a	b	c	z
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Specifically:

1. Examine the truth table for all combinations where the output is 1.
2. Write down the specific combination of inputs where the output is 1.
3. Where there are multiple, `OR` all of the combinations from above to form a SOP expression.
4. Simplify.

Minterms

Terms that are written solely as `AND`s of all inputs or their inverses are called *minterms*. All inputs must appear in the expression exactly once, although they can be inverted.

Term	Minterm
abc	
$\bar{a}bc$	
$\bar{a}b$	
b	
$a(b+c)$	

Minterms specifically will give us the SOP form of an expression, thus the SOP form is also known as the sum of minterms, or minterm canonical form.

3.2 Simplification

Typically, we will be using the Distributive, Or-Complement, and Identity rules. It is also worth noting that we can duplicate minterms as per idempotence. Some tips:

- Recall that everything we are dealing with is a 0 or 1
- Order of operations: `NOT`, then `AND`, and finally `OR`.

Using the above example:

$$z = \bar{a}\bar{b}c + a\bar{b}\bar{c} + a\bar{b}c + abc$$

We can then double check our answers with another truth table. Also, note how we represent "don't care" inputs:

a	b	c		z
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

3.3 Logic gates

Typically, a logic gate is used for each Boolean operation within an expression. The logic gates that are common are:

- `AND`
- `OR`
- `NOT`

Recall:

- The small bubble = `NOT`, and may be incorporated into the gate, at the output **or the input**.
- Gates can contain more than one input.

3.3.1 Conversion from an expression

Boolean expressions can be thought of as a "tree", which converges at the top to the overall expressions. The shape of the tree shows us the order of operations, and therefore gives the shape of the overall logic circuit: For example: $z = a(b + c)$:

Summary

- Understand logic statements, variables, and values:
  → Logic statements are sentences that answer a yes/no question.
  → Logic variables are the programming variable names in which we store the answers.
  → Logic values are the values stored within the above variables: either 0/1, `FALSE`/`TRUE`, or the physical representation of how this electrically achieved.
- Understand logic functions.
  → Logic functions are functions which have logical inputs an outputs, just as numerical functions take numerical inputs and outputs.
  → The *fundamental* logic functions are `NOT`, `OR`, and `AND`.
- Understand and use tools to simplify logic functions.
  → There are several algebraic tools that we can use to reduce logic equations to more simplistic ones.
  → De Morgan's laws specifically are useful because they allow reduction to a single operation.

Next Lecture: Lecture 5 - CPU

4 Questions?